Tutorial: evolución de ByteMadrid de la fase 1 a la fase 2

Profesor: Fernando Sánchez Martínez · Correo: fsanchezmartinez1@educa.madrid.org
Entorno: Odoo 18 Community, Python, XML, ORM y PostgreSQL · Duración: 45–60 minutos

1. Objetivo de la práctica

En la fase 1 incorporamos las relaciones proveedor_id y responsable_id. En la fase 2 añadiremos un estado operativo y seis botones que ejecutan métodos Python. El objetivo es aprender a implementar reglas de negocio y controlar cambios de estado.

2. Crear el campo de estado en Python

Abre addons/bytemadrid_equipos/models/equipo.py. Comprueba que el archivo importa también UserError:

```
from odoo import models, fields
from odoo.exceptions import UserError
```

Dentro de la clase Equipo, añade el campo:

```
estado = fields.Selection(
    selection=[
        ('disponible', 'Disponible'),
        ('asignado', 'Asignado'),
        ('averiado', 'Averiado'),
        ('baja', 'Baja'),
    ],
    string='Estado operativo',
    required=True,
    default='disponible',
)
```

Explicación: Selection limita las opciones posibles; cada pareja contiene el valor técnico almacenado y la etiqueta visible. required=True obliga a indicar un estado y default asigna «disponible» inicialmente. El campo activo de la fase 1 sigue existiendo y no equivale a «baja».

3. Programar botones con métodos Python

Añade dentro de la misma clase el método siguiente (respetando la indentación):

```
def action_asignar(self):
    for equipo in self:
        if not equipo.responsable_id:
            raise UserError('Selecciona un responsable antes de asignar el equipo.')
        if equipo.estado != 'disponible':
            raise UserError('Solo se puede asignar un equipo disponible.')
        equipo.write({'estado': 'asignado'})
    return True
```

Lectura: self representa los registros; el bucle procesa cada equipo; if comprueba condiciones; UserError impide una acción incorrecta; write() guarda el cambio mediante el ORM, sin escribir SQL manualmente.

4. Completar las transiciones

El módulo incorpora otros cinco métodos con el mismo patrón de validación y escritura:

Método	Cuándo se permite	Resultado
action_liberar	Asignado	Disponible; limpia responsable
action_marcar_averiado	Disponible o asignado	Averiado
action_reparar	Averiado	Disponible; limpia responsable

Método	Cuándo se permite	Resultado
action_dar_baja	Cualquiera excepto baja	Baja; limpia responsable
action_reactivar	Baja	Disponible

5. Conectar los botones en XML

Abre views/equipo_views.xml. Dentro del formulario, coloca un <header> con botones como estos:

```
<header>
  <button name="action_asignar" type="object"
          string="Asignar" invisible="estado != 'disponible'"/>
  <button name="action_liberar" type="object"
          string="Liberar" invisible="estado != 'asignado'"/>
  <field name="estado" widget="statusbar"
         statusbar_visible="disponible,asignado,averiado,baja"
         readonly="1"/>
</header>
```

type="object" llama al método Python indicado en name. invisible oculta el botón cuando no procede, pero no es una medida de seguridad: la validación debe seguir en Python. statusbar muestra los estados en la cabecera. Añade los otros cuatro botones con sus nombres y condiciones, como en el módulo facilitado por el profesor.

6. Mostrar el estado en el listado

En la vista <list> incorpora:

```
<field name="estado" widget="badge"
       decoration-success="estado == 'disponible'"
       decoration-info="estado == 'asignado'"
       decoration-warning="estado == 'averiado'"
       decoration-danger="estado == 'baja'"/>
```

El widget badge muestra una etiqueta; las decoraciones aplican estilos visuales según el valor del estado. En Odoo 18 la vista de lista se define con <list>.

7. Actualizar y comprobar

Haz una copia de seguridad. Sustituye los archivos del módulo, reinicia Odoo con docker restart odoo y entra en Aplicaciones → ByteMadrid – Equipos → Actualizar. No desinstales el módulo ni borres volúmenes: queremos conservar los registros anteriores.

8. Pruebas obligatorias

Crea «PC Aula 02» sin responsable e intenta Asignar: debe mostrar un error. Selecciona un usuario interno y vuelve a asignar: pasa a Asignado. Prueba después Liberar, Registrar avería, Reparado, Dar de baja y Reactivar. Recarga la página y comprueba que los cambios persisten.

```
SELECT id, name, estado, responsable_id
FROM bytemadrid_equipo ORDER BY id;
```

9. Actividad de entrega

Entrega un único PDF con: (1) captura de la barra de estados, (2) captura de un error al asignar sin responsable, (3) captura del equipo asignado, (4) explicación de Selection, write(), UserError y type="object", y (5) un diagrama sencillo con los cuatro estados y las transiciones permitidas.

Preguntas: ¿Por qué ocultar un botón no impide ejecutar su método desde otro cliente? ¿Qué diferencia hay entre «baja» y el campo «activo»? ¿Por qué poner responsable_id=False no elimina al usuario de Odoo?

Nota docente: este tutorial explica el cambio principal. Para copiar íntegramente los seis métodos y botones, utiliza el módulo de la fase 2 entregado por el profesor.